50 Stars | 5 million Evaulations per trial | 32 Trials | 50% Success Rate

Stellar Oscillation Optimization

—

By Jacob Hall-Burns, Conor Shanley, Brice Vankregten

Sudoku

- 9x9 grid with nine 3x3 subgrids
- Each row, column and subgrid must contain 1-9 exactly once
- Pre given clues are immutable
- It is estimated that there are approximately 6.67x10^21 valid sudoku grids
- When a problem has many empty cells, the solution approaches NP-hard constraint satisfaction

Metaheuristics

- Exact solvers can be too slow for hard puzzles
- Metaheuristics provide “good enough” solutions within a budget
- Problem-agnostic: same framework, many problems

We choose Stellar Oscillation Optimization (SOO, 2025)

Stellar Oscillation Optimization

Inspiration

SOO is based on Asteroseismology, the study of pulsating stars. Stars oscillating expansion and contraction can be measured to reveal their internal structure.

Exploration (Sine)/ Exploitation (Cosine)

The exploration and exploitation of the SOO metaheuristic is driven by the sine and cosine wave function. Exploration caused the “star” to pulsate, randomizes subgrids based on intensity and a conflict heatmap. Exploitation contracts the “star” toward the current “best star”s board state.

Algorithm

Solution Representation

81 character list. Subgrid validity guaranteed by construction of population.

Exploration

Sub grids are selected by a heat map of conflicts. Intensity is proportionaly to the absolute value of sin(φ). Swaps or triple-rotations of non-fixed cells mirror sine-driven expansion.

Event Horizon Trigger

Global stagnation exceeds 10% of budget. Population is reinitialized, with ~30% of the “stars” being completely randomized amd 70% being seeded from the global best.

Design

Exploitation

Copies values from X_best into different cells within the same subgrid. Intensity is proportional to the cosine of φ. Stars near X_best use reduced intensity.

Supernova

Triggered when stagnation count exceeds 64, which represents a full 2π cycle through sine/ cosine. Stars are reseeded from the global best, with 15 permutations based on the conflict heat map.

Fitness - Luminosity

$L = 1/(1+\text{conflicts})$

Phase Schedule

Phase Schedule of φ

$\Delta\varphi = \max(0.01, 0.1(1- (\text{evaluations} / \text{budget}))$

Early on, Δφ is fast and rapidly oscillates between exploration and exploitation.

Later on, Δφ is slow and there is a focus on exploitation near the best solution.

Star Behavior by State

Best Star - Target pulsation only

Low Conflict (<15) - Heavy contraction, light pulsation

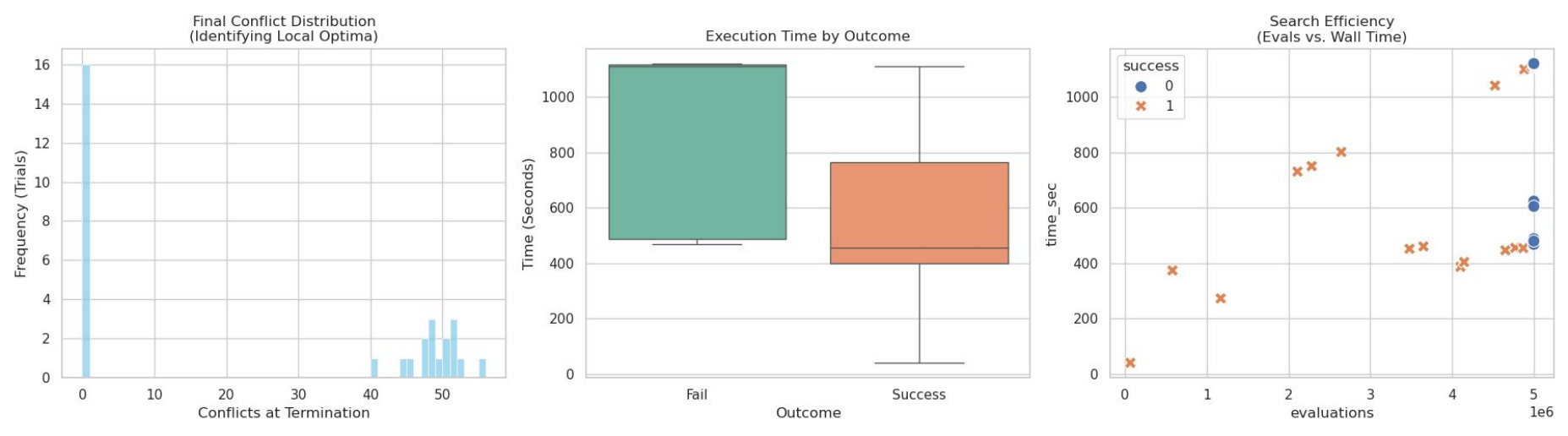
High Conflict (>= 15) - Balanced pulsation and contraction

```python
def main():
    # Evaluates the input for being valid, and throws an error if it is not.
    # Creates a list of the fixed indices
    try:
        initial_board = sys.stdin.readline().strip()
        max_solutions_explored = int(sys.stdin.readline().strip())
        initial_validation(initial_board, max_solutions_explored)
        fixed_cells = [k for k, val in enumerate(initial_board) if val != "."]
    except (ValueError, EOFError) as e:
        print(f"Input error: {e}", file=sys.stderr)
        sys.exit(1)

    # Default population size is set to 50. This is arbitrary
    # The constellation of stars is initialized
    population_size = 50
    constellation = [Star(population_initialization(initial_board), fixed_cells)
                     for _ in range(population_size)]

    # The Event Horizon Trigger is set at 10% of budget, capped at 500k
    event_horizon_trigger = min(500000, max(5000, max_solutions_explored // 10))

    # Evaluations are initialized
    evaluations = 0

    # First check through the randomly subgrid valid Sudoku boards to see if a "perfect" star was found on initiation
    for star in constellation:
        star.evaluate()
        evaluations += 1
        if star.conflict_count == 0:
            print(f"{''.join(star.board)}")
            print(f"Solutions Explored: {evaluations}")
            print("Conflicts: 0")
            return
        """
        # Submission Version
        if star.conflict_count == 0:
            print(f"{''.join(star.board)}\nConflicts: 0")
            return
        """
    # The global best star is created based on the first star in the constellation index with the worst number of conflicts
    global_best_star = min(constellation, key=lambda s: s.conflict_count)


    # Global Stagnation is initiated
    global_stagnation = 0

    # Phi is initialized
    phi = 0.0
```

```python
    # While the number of evaluations is less than the budget, keep going
    while evaluations < max_solutions_explored:
        # The current best is set
        X_best = max(constellation, key=lambda s: s.luminosity)
        # The tick of the pulsation and contraction is based on the absolute value of the sin and cosine of phi
        T_pulsate = abs(math.sin(phi))
        T_contract = abs(math.cos(phi))

        # If the global stagnation is greater than 10% of the budget, trigger the event horizon
        if global_stagnation > event_horizon_trigger:
            for i in range(population_size):
                # Leave the current best alone
                if constellation[i] != X_best:
                    # Chance for the star to be reinitialized from scratch
                    if random.random() < 0.3:
                        constellation[i] = Star(population_initialization(initial_board), fixed_cells)
                    # Otherwise, a supernova is executed against the star, with reference to the global_best_star
                    else:
                        constellation[i] = Star(list(global_best_star.board), fixed_cells)
                        constellation[i].supernova_reset(global_best_star)
            # Global_stagnation and phi are reset
            global_stagnation, phi = 0, 0.0

```

```python
        # For each star we pulsated then contract before evaluating the star
        for star in constellation:
            # Initialized the values for each tick of the global clock
            max_int = 9 if star.conflict_count > 20 else 3
            # Phi changes every time an evaluation cycle finishes. This pushes us through the peaks and valleys of sin and cosine waves overtime
            int_p, int_c = round(T_pulsate * max_int), round(T_contract * max_int)

            # If the current star is the current best, there is a low chance to pulsate the star at a low intensity
            if star == X_best:
                if random.random() < T_pulsate: star.targeted_pulsation()
            # If the star has less than 15 conflicts
            elif star.conflict_count < 15:
                # Chance for a really small pulsation
                if random.random() < 0.05: star.pulsate(1)
                # Contract towards X_best with high intensity
                star.contract_toward(X_best, intensity=9)
            # Otherwise, pulsate at the calculated intensity and contract towards X_best at the calculated intensity
            else:
                star.pulsate(int_p)
                star.contract_toward(X_best, int_c)
            # Each star is evaluated
            star.evaluate()
            evaluations += 1
            # If the star has no errors we can finish the program and escape
            if star.conflict_count == 0:
                print(f"{''.join(star.board)}")
                print(f"Solutions Explored: {evaluations}")
                print("Conflicts: 0")
                return
            """
            # Submission Version
            if star.conflict_count == 0:
                print(f"{''.join(star.board)}\nConflicts: 0")
                return
            """
            # If the star's luminosity is better than the current global best, we replace it
            if star.luminosity > global_best_star.luminosity:
                global_best_star.board, global_best_star.conflict_count = list(star.board), star.conflict_count
                global_best_star.luminosity, global_stagnation = star.luminosity, 0
            # If not, we increase the global stagnation (which is different than the stagnation_count)
            else:
                global_stagnation += 1
            # If the stagnation count for a given star is greater than 64, supernova_reset.
            # 64 is approximately one full oscillation cycle at the initial phase increment rate.
            if star.stagnation_count > 64:
                star.supernova_reset(global_best_star)
        # Phi is increased
        phi += max(0.01, 0.1 * (1 - (evaluations / max_solutions_explored)))
```

Demo



Results

- 50% Success Rate! - 16/32 Trials Solved
- ~4.14 million Average Evaluations
- 5 Million Evaluations per Trial

Complexity and SOO mapping

Complexity

- Evaluations per board - O(1) (162 cells)
- Population - O(N) - N =50 stars
- Full Run - O(B) - B = Budget
- Space - O(N) ~= 50x81 cells

Known Limitations

- Python is slow (~600-3000s per trial)
- Parameters are not tuned via ablation

Original SOO	Our Implementation
Real-value position update	Within subgrid swap, triple-rotate
Sine-driven expansion	Conflict-heat pulsation
Cosine-driven contraction	Contract toward X_best (local best)
Average top-3 oscillators	X_best guides population
Oscillation Period P(t)	Phase counter φ schedule
Scaling factor S(t)	Max_int * |sin/cos(φ)|
No stagnation mechanism	Supernova reset + Event Horizon

Conclusion and Future Work

Accomplished

- Adapted SOO's exploration-exploitation duality to a discrete, permutation-constrained domain
- Preserved subgrid validity by construction
- No constraint repairs needed
- 50% success rate on a (hard-expert)-difficulty puzzle
- Full 32-trial parallel benchmark with analysis pipeline

Future Directions

- Compiled implementation in C or Rust for trial speedup
- Ablation study on stagnation thresholds
- Compare vs SA and GA on the same puzzle

Questions?